

Министерство науки и высшего образования Российской Федерации  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ  
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО  
(НИУ ИТМО)

Факультет программной инженерии и компьютерной техники

ОТЧЁТ  
О ЛАБОРАТОРНОЙ РАБОТЕ № 1  
Проектирование вычислительных систем  
по теме:  
Интерфейсы ввода-вывода общего назначения (GPIO)  
Вариант 3

Выполнили: студенты группы Р3434

Р. Д. Баянов

Д. А. Кузнецов

Проверил:

преподаватель В. Ю. Пинкевич

Санкт-Петербург 2025

## СОДЕРЖАНИЕ

|                                                                                                   |    |
|---------------------------------------------------------------------------------------------------|----|
| Цель работы .....                                                                                 | 3  |
| Задание .....                                                                                     | 4  |
| Вариант .....                                                                                     | 5  |
| 1 Описание организации программы и структуры драйверов, блок-<br>схема прикладного алгоритма..... | 6  |
| 2 Исходный код .....                                                                              | 8  |
| ЗАКЛЮЧЕНИЕ.....                                                                                   | 10 |

## **ЦЕЛЬ РАБОТЫ**

1. Получить базовые знания о принципах устройства стенда SDK-1.1M и программировании микроконтроллеров.
2. Изучить устройство интерфейсов ввода-вывода общего назначения (GPIO) в микроконтроллерах и приемы использования данных интерфейсов.

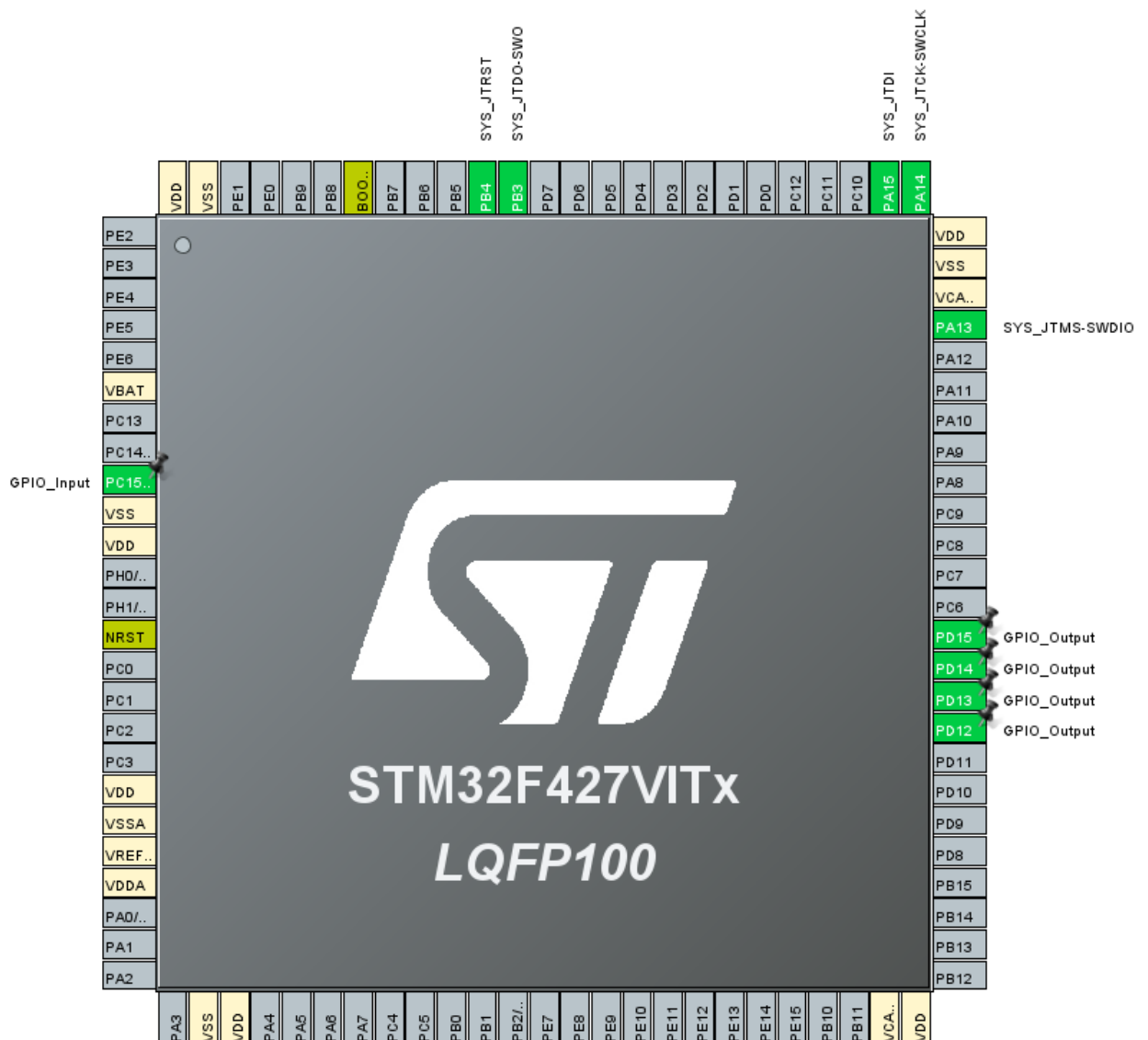
## ЗАДАНИЕ

Разработать и реализовать драйверы управления светодиодными индикаторами и чтения состояния кнопки стенда SDK-1.1M (расположены на боковой панели стенда). Обработка нажатия кнопки в программе должна включать программную защиту отдребезга. Функции и другие компоненты драйверов должны быть универсальными, т. е. пригодными для использования в любом из вариантов задания и не должны содержать прикладной логики программы. Функции драйверов должны быть неблокирующими, то есть не должны содержать ожиданий события (например, нажатия кнопки). Также, в драйверах не должно быть пауз с активным ожиданием (функция `HAL_Delay()` и собственные варианты с аналогичной функциональностью). При необходимости параллельного выполнения разных процессов (например, опроса кнопки и ожидания перед переключением светодиодов) следует организовать псевдопараллельную работу процессов в стиле кооперативной многозадачности. Это возможно сделать без существенной потери точности соблюдения необходимых интервалов времени между действиями, поскольку действия выполняются очень быстро по сравнению с промежутками ожидания между ними. Каждый процесс (который может быть выражен всего в нескольких строках кода) должен использовать не активное ожидание (`HAL_Delay()`), а считывать текущее значение миллисекундного счетчика (`HAL_GetTick()`), проверяя, прошло ли необходимое количество времени для выполнения следующего действия. После проверки и (при необходимости) выполнения действия управление должно передаваться следующему процессу, чтобы он тоже мог провести такую проверку. Контакты подключения кнопки и светодиодов должны быть настроены в режиме GPIO. Использование прерываний и дополнительных таймеров (кроме `SysTick`) не допускается. Написать программу с использованием разработанных драйверов в соответствии с вариантом задания.

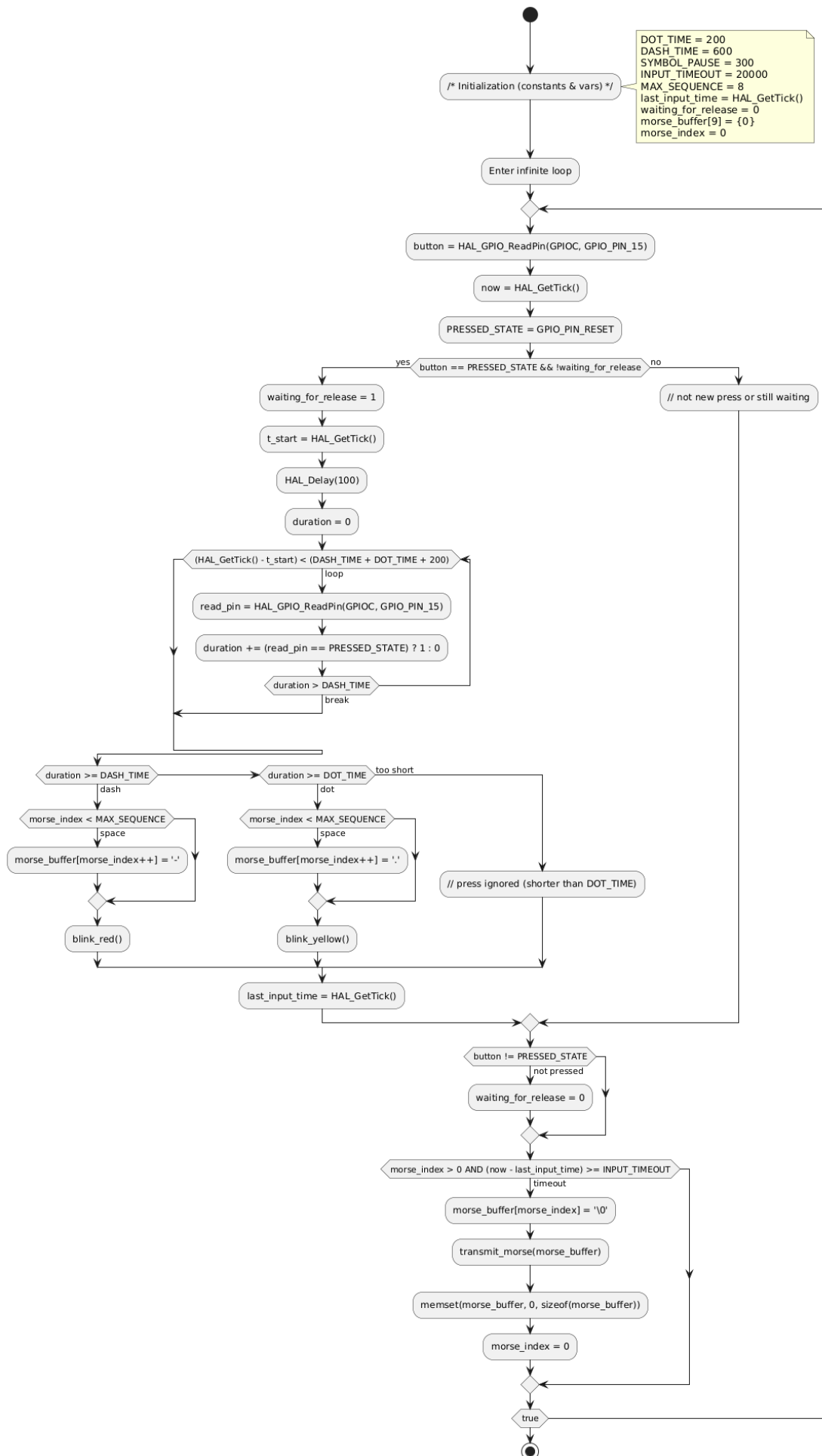
## **ВАРИАНТ**

Реализовать «передатчик» азбуки Морзе. Программа должна запоминать последовательность нажатий кнопки (короткое – точка, длинное – тире) длиной до 8 нажатий. При этом после каждого нажатия кнопки двухцветный светодиод должен коротким 76 мерцанием показывать, какой сигнал был введен: точка – желтым, тире – красным. После окончания ввода (долгая пауза после последнего нажатия кнопки) последовательность один раз «отправляется» при помощи зелёного светодиода быстрыми (точка) и долгими (тире) импульсами. После этого программа снова переходит в режим ввода последовательности.

# 1 Описание организации программы и структуры драйверов, блок-схема прикладного алгоритма



## 1.1 Блок-схема



## 2 Исходный код

---

```
const uint32_t DOT_TIME = 200;
const uint32_t DASH_TIME = 600;
const uint32_t SYMBOL_PAUSE = 300;
const uint32_t INPUT_TIMEOUT = 2000;
const int MAX_SEQUENCE = 8;

uint32_t last_input_time = 0;
_Bool wait_for_release = 0;

char morse_buffer[9] = {0};
int morse_index = 0;
```

---

```
while (1)
{
    GPIO_PinState button = HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_15);
    uint32_t now = HAL_GetTick();
    const GPIO_PinState PRESSED_STATE = GPIO_PIN_RESET;
    if (button == PRESSED_STATE && !wait_for_release) {
        uint32_t t_start = HAL_GetTick();
        HAL_Delay(100);
        while (PRESSED_STATE == HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_15)
&& (HAL_GetTick() - t_start) < (DASH_TIME + DOT_TIME + 200)) {
            uint32_t duration = HAL_GetTick() - t_start;
            if (duration >= DASH_TIME) {
                if (morse_index < MAX_SEQUENCE)
morse_buffer[morse_index++] = '-';
                blink_red();
            } else if (duration >= DOT_TIME) {
                if (morse_index < MAX_SEQUENCE)
morse_buffer[morse_index++] = '.';
                blink_yellow();
            }
            last_input_time = HAL_GetTick();
        }
        if (PRESSED_STATE != HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_15)) {
            wait_for_release = 0;
        }
        int diff = (int) now - (int) last_input_time;

        if (morse_index == MAX_SEQUENCE || (diff >= (int)
INPUT_TIMEOUT && morse_index > 0)) {
            morse_buffer[morse_index] = '\\0';
            transmit_morse(morse_buffer);
            memset(morse_buffer, 0, sizeof(morse_buffer));
            morse_index = 0;
        }
    }
}
```

---



---

```
void blink_yellow(void) {
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, GPIO_PIN_SET);
    HAL_Delay(150);
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, GPIO_PIN_SET);
}

void blink_red(void) {
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_SET);
    HAL_Delay(300);
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_SET);
}

void transmit_morse (char *sequence) {
    int dot_time = 200;
    int dash_time = 600;
    int symbol_pause = 300;

    for (int i = 0; i < strlen(sequence); i++) {

        if (sequence[i] == '.') {
            HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, GPIO_PIN_SET);
            HAL_Delay(150);
            HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, GPIO_PIN_RESET);
        } else if (sequence[i] == '-') {
            HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, GPIO_PIN_SET);
            HAL_Delay(500);
            HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, GPIO_PIN_RESET);
        }
        HAL_Delay(symbol_pause);
    }
}
```

---

## **ЗАКЛЮЧЕНИЕ**

В ходе работы удалось настроить и запрограммировать микроконтроллер STM32F427VIT6 для работы с вводом-выводом GPIO.